

# Take-Home Data Engineering Assignment

---

A realistic ELT take-home with a runnable reference solution

ingest → clean → star schema → data quality → analytics → tests (+ Spark/AWS scale-up)

A realistic take-home you can practice with, plus a **working reference solution** you can run. Two parts: the **assignment** (what a company hands you) and the **solution** (how to do it well, including how you'd build the same thing at scale on AWS/Spark).

---

## PART 1 — The assignment (the prompt)

**Scenario:** You're a data engineer at an online retailer. You've been given three raw files: `customers.csv`, `products.csv`, and `orders.csv`. The orders data is messy — it has duplicates, missing amounts, cancelled/pending orders, and some orders referencing customers or products that don't exist.

>

### Build a pipeline that:

1. **Ingests** the three raw files.
2. **Cleans** the orders: keep only *completed* orders with a valid positive amount, remove duplicates, and drop orders with invalid customer/product references.
3. **Models** the data into a **star schema**: `dim_customer`, `dim_product`, `dim_date`, and `fact_orders`.
4. **Validates** the output with **data-quality checks** (uniqueness, no nulls in keys, positive amounts, referential integrity) that *fail the run* if broken.
5. **Produces analytics**: revenue by region, top products, and daily revenue.
6. Includes **tests** and a short **write-up** of your design decisions.

>

**Evaluation:** correctness, code quality/readability, data-quality thinking, testing, and how you'd scale this to millions of rows in the cloud.

(This mirrors real take-homes — the twist is always "the data is dirty; show me you handle it and you think about quality and scale.")

---

## PART 2 — The reference solution

### How to run

```
python3 generate_data.py      # creates messy raw CSVs in data/raw/
python3 pipeline.py          # ELT: clean -> star schema -> data-quality checks -> analytics
python3 test_pipeline.py      # tests (gate for CI)
```

Pure standard library (SQLite) — runs anywhere, no installs.

## Project layout

```
de-take-home-project/
├─ generate_data.py      # makes dirty sample data (dups, nulls, cancelled, bad refs)
├─ pipeline.py           # the ELT pipeline (bronze → silver → gold star schema + DQ +
analytics)
├─ test_pipeline.py      # assertions (row counts, dedup, reconciliation, integrity)
├─ data/raw/             # raw input CSVs
├─ data/curated/         # output star-schema tables (CSV; "the gold zone")
└─ warehouse.db          # the SQLite "warehouse"
```

## Pipeline flow (medallion pattern)

```
RAW csv → staging (bronze, as-is) → cleaned_orders (silver) → star schema (gold)
                                                    dim_customer /
dim_product
                                                    dim_date / fact_orders
                        then: data-quality checks + analytics
```

## The cleaning rules (the heart of it)

Applied in `cleaned_orders`:

- **Type the amount** ( `'` → NULL → REAL), drop NULL/ `<= 0`.
- **Deduplicate** by `order_id` keeping the latest timestamp — `ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY order_ts DESC) = 1` (the classic CDC/upsert dedup).
- **Keep only completed** orders.
- **Referential integrity**: inner-join to customers and products so orphan references are dropped.

Result on the sample run: **2,030 raw rows** → **1,354 clean orders**.

## The star schema (dimensional model)

- **Fact**: `fact_orders(order_id, customer_id, product_id, date_key, quantity, amount)` — the grain is one completed order line.
- **Dimensions**: `dim_customer` (region defaults to *Unknown* if blank), `dim_product` (category, price), `dim_date` (year/month/day derived from the timestamp). This is the shape BI tools want: sum the fact, group/filter by dimensions.

## Data-quality checks (fail-the-pipeline)

`pipeline.py` runs and **exits non-zero** if any fails:

- `order_id` unique in the fact,

- no nulls in keys/amounts,
- all amounts > 0,
- every fact row's customer & product exist in their dimension.

## Tests

`test_pipeline.py` asserts: fact has rows, only completed orders survived, no duplicate ids, **revenue reconciles** (fact total == cleaned total), and date integrity — a gate you'd wire into CI.

---

## PART 3 — How you'd build this at scale (talk about this!)

The reference solution uses SQLite so it runs anywhere, but in a real role you'd say:

### On AWS / Spark:

```
# same logic in PySpark
from pyspark.sql import functions as F, Window

orders = spark.read.csv("s3://.../raw/orders/", header=True, inferSchema=True)

w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
clean = (orders
    .withColumn("amount", F.col("amount").cast("double"))
    .filter((F.col("status") == "completed") & (F.col("amount") > 0))
    .withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
    .join(F.broadcast(customers), "customer_id")      # integrity + small-dim broadcast
    .join(F.broadcast(products), "product_id"))

fact = clean.withColumn("date_key", F.to_date("order_ts")) \
    .select("order_id", "customer_id", "product_id", "date_key", "quantity", "amount")
fact.write.mode("overwrite").partitionBy("date_key").parquet("s3://.../curated/fact_orders/")
```

- **Storage:** Parquet in S3, partitioned by date (medallion zones raw→clean→curated).
  - **Compute:** Glue/Databricks (Spark); broadcast the small dims; incremental via **job bookmarks** or a watermark; upserts via **Delta/Iceberg MERGE**.
  - **Catalog + serve:** Glue Crawler → Athena/Redshift/Snowflake; dashboards on top.
  - **Quality:** dbt tests / Great Expectations instead of the hand-rolled checks.
  - **Orchestrate:** Airflow/Step Functions with retries + alerts; event-trigger on new files.
  - **IaC + CI/CD:** Terraform for infra; Git + CI runs the tests before deploy.
-

## PART 4 — What evaluators look for (score yourself)

| Criteria             | What good looks like                                               |
|----------------------|--------------------------------------------------------------------|
| Correctness          | Cleaning rules applied; counts make sense; analytics are right     |
| Handles dirty data   | Dedup, nulls, bad refs, wrong status all dealt with explicitly     |
| Data quality         | Checks that <i>fail</i> the run; reconciliation                    |
| Modeling             | Proper star schema; clear grain; sensible dimensions               |
| Code quality         | Readable, modular, commented; no hard-coding                       |
| Testing              | Meaningful assertions, CI-gate-ready                               |
| Scalability thinking | Can explain the Spark/AWS version, partitioning, incremental, cost |
| Communication        | A short, clear write-up of decisions and trade-offs                |

**Interview tip:** even when the take-home is small, say how you'd scale it — that's what separates a mid from a senior. Always mention **idempotency, partitioning, incremental loads, data quality, and orchestration**.